Servis Defteri Entegrasyon API

Sürüm 1.1 · Makine-makine (server-to-server) REST API · JSON · HTTPS · Temel adres: `https://alanadiniz.com/api/v1`

> Bu belgedeki `alanadiniz.com`, Servis Defteri'ni işleten işletmenin alan adıyla değiştirilir. Adresi ve API anahtarını işletme size iletir.

1. Genel bakış

Servis Defteri'nde atölye çalışanı tarafından açılmış servis (iş emri) kaydı, harici bir sistemden (ör. muhasebe yazılımı) gelen bilgiyle eşleştirilir ve güncellenir. API yeni servis kaydı açmaz: gönderilen plakaya ait tek açık iş emrini bulur ve bilgiyi ona işler. Eşleşme güvenli değilse kayıt kalıcı olarak saklanır ve insan incelemesine alınır; hiçbir kayıt tahminle bağlanmaz.

API'nin alanları Servis Defteri'nin kendi veri modeliyle birebir aynıdır (müşteri, araç, iş emri).

Akış

1. Atölye çalışanı aracı Servis Defteri'nde plaka ve km ile kaydeder (iş emri açılır, etiket basılır).
2. Muhasebe yazılımı servisi işleyip kaydı oluşturunca tek bir `POST /is-emirleri` gönderir.
3. Servis Defteri kaydı plakadaki açık iş emriyle eşleştirir; "Yapılan işlem", durum ve eksik müşteri/araç bilgilerini günceller. Etiket yeniden basılmaz, yeni iş emri açılmaz.

2. Kimlik doğrulama

Her istekte API anahtarı gönderilir (anahtar ayrı ve güvenli bir kanaldan iletilir; e-posta/mesajlaşma ile açık yazılmamalıdır):

```
Authorization: Bearer <API_ANAHTARI>              (alternatif: X-API-Key: <API_ANAHTARI>)
```

Art arda 10 hatalı anahtar denemesinde ilgili IP için 5 dakika 429 döner (`Retry-After: 300`). Anahtar sızarsa yenisi üretilir; eski anahtar o anda geçersiz olur.

3. HTTPS ve istemci gereksinimleri

API yalnızca HTTPS (TLS 1.2 veya üzeri) ile kullanılır. Anahtar her istekte gönderildiği için düz `http://` güvenli değildir; düz http ile gelen istekler `400 HTTPS_REQUIRED` ile reddedilir. İstemci yazılımınız yönlendirmeleri (301/302) POST olarak takip etmeyebilir; adresi doğrudan `https://` ile yapılandırın.

> **Önemli — User-Agent başlığı:** Sunucunun önünde bir güvenlik duvarı (ör. Cloudflare) çalışıyor olabilir. `User-Agent` başlığı olmayan ya da kütüphane varsayılanını (ör. `Python-urllib`) taşıyan istekler bot sanılıp 403 ile engellenebilir; bu durumda yanıt gövdesi boş ya da JSON olmaz. İsteklerinize anlamlı bir kimlik ekleyin, örn. `User-Agent: MuhasebeYazilimi/1.0`. Sabit çıkış IP'niz varsa işletmeye bildirin; gerekirse izin listesine eklenir.

4. Uç noktalar

Yöntem	Yol	Açıklama
POST	`/is-emirleri`	Servis kaydı bildirimi (eşleştirme ve güncelleme)
GET	`/saglik`	Sağlık kontrolü (anahtar gerekmez, veri döndürmez)

5. POST /is-emirleri — istek alanları

Yalnızca `plaka` zorunludur; diğer alanların hepsi isteğe bağlıdır. İçerik türü `application/json` (UTF-8).

Alan	Tür	Açıklama
Kayıt		
`kayit_no`	string ≤64	Karşı sistemdeki kayıt/belge numarası. Yinelenmeyi önler (idempotency anahtarı). Önerilir. Gönderilmezse yalnızca birebir aynı içerikli istek tekrar sayılır (sınırlı koruma).
Araç		
`plaka`	string	Zorunlu. `35 ABC 123` / `35ABC123`. Özel/yabancı plaka için `ozel_plaka: true`.
`km`	integer	Eşleştirmeyi güvenli kılar (varsayılan ±50 km tolerans). Kayıtlı giriş km'sinin yerine geçmez.
`marka`, `model`	string ≤40	Yalnızca boşsa tamamlanır.
Müşteri		
`musteri_ad`	string ≤100	Yalnızca isimsiz (İSİMSİZ) kayıtta güncellenir.
`telefon`, `eposta`, `adres`	string	Yalnızca boşsa tamamlanır. Geçersiz telefon/e-posta biçimi yok sayılır.
İş emri		
`yapilan_islem`	string | string[]	Yapılan işlemler. Karşı sistem son karardır: mevcut "Yapılan işlem" alanının yerine yazılır; eski metin kaybolmaz, nota "Önceki işlem: …" olarak saklanır. Gönderilmezse işlem alanı değişmez ve işlem uydurulmaz.
`durum`	string	`Beklemede` | `Devam Ediyor` | `Tamamlandı`. Gönderilirse iş emri durumu olur; Tamamlandı ise gidiş zamanı da yazılır. Gönderilmezse durum değişmez.
`gelis_zamani`, `gidis_zamani`	ISO 8601	Örn. `2026-10-22T14:30:00+03:00` veya `…Z`; saat dilimsiz değer Türkiye saati sayılır. `gidis_zamani`, durum Tamamlandı iken gidiş zamanı olur (yoksa şimdiki zaman).

`not`	string ≤1000	İş emri notuna eklenir.

"Ne yapılacak?" (geliş nedeni) alanı API'den değiştirilemez; bu bilgiyi atölye çalışanı Servis Defteri'nde girer.

En basit istek

```
POST /api/v1/is-emirleri
Authorization: Bearer <API_ANAHTARI>
User-Agent: MuhasebeYazilimi/1.0
Content-Type: application/json

{ "plaka": "35ABC123", "km": 120000 }
```

Yalnızca plaka ve km gönderilirse mevcut iş emri eşleştirilir; işlem bilgisi uydurulmaz.

Tam istek örneği

```
curl -X POST https://alanadiniz.com/api/v1/is-emirleri \
  -H "Authorization: Bearer <API_ANAHTARI>" -H "Content-Type: application/json" \
  -H "User-Agent: MuhasebeYazilimi/1.0" \
  -d '{"kayit_no":"K-10425","plaka":"35 ABC 123","km":120010,
       "yapilan_islem":["Yağ değişimi","Yağ filtresi"],"durum":"Tamamlandı",
       "gidis_zamani":"2026-10-22T17:00:00+03:00","not":"Fatura F-55"}'
```

6. Yanıtlar

HTTP	Anlamı	Örnek gövde
200	Mevcut iş emriyle eşleşti ve güncellendi	`{"ok":true,"eslesti":true,"durum":"eslesti","is_emri_id":12}`
200	Aynı `kayit_no` daha önce işlendi (değişiklik yok)	`{"ok":true,"tekrar":true,"durum":"eslesti"}`
202	Güvenle eşleşemedi; kalıcı olarak kaydedildi, insan incelemesine alındı. **Yeniden göndermeye gerek yoktur.**	`{"ok":true,"eslesti":false,"durum":"beklemede","neden":"…"}`
400	Düz http ile gelen istek	`{"ok":false,"kod":"HTTPS_REQUIRED"}`
401	Geçersiz/eksik API anahtarı	`{"ok":false,"kod":"INVALID_API_KEY"}`

403	Uygulamadan değil, önündeki güvenlik duvarından gelir (gövde JSON değil/boş). User-Agent ekleyin (bkz. Bölüm 3).	—
404	Entegrasyon bu sunucuda etkin değil	—
409	(a) `IDEMPOTENCY_CONFLICT`: aynı `kayit_no` daha önce farklı içerikle işlenmişti; kayıt değiştirilmedi, otomatik tekrar denenmemeli. (b) `CONFLICT`: aynı kayıt eş zamanlı işleniyor; birkaç saniye sonra tekrar gönderilebilir.	`{"ok":false,"kod":"IDEMPOTENCY_CONFLICT",…}`
422	Eksik/geçersiz alan (plaka, durum, zaman…)	`{"ok":false,"kod":"VALIDATION_ERROR","hata":"…"}`
429	Çok fazla hatalı anahtar denemesi	`{"ok":false,"kod":"RATE_LIMITED"}`
500	Geçici sunucu hatası — aynı istek güvenle tekrar denenebilir	`{"ok":false,"kod":"SERVER_ERROR"}`

Hata yönetimi önerisi

Durum	İstemci davranışı
200, 202	Başarılı sayın; tekrar göndermeyin
500, 502, 503, 504, ağ hatası	Üstel bekleme ile aynı isteği (aynı `kayit_no`) tekrar deneyin
429	`Retry-After` başlığındaki süre (sn) kadar bekleyin
400, 401, 403, 422, 409 (IDEMPOTENCY_CONFLICT)	Otomatik tekrar denemeyin; önce nedeni düzeltin

7. Eşleştirme kuralları

Durum	Davranış
Plaka kayıtlı, tek açık iş emri, km uygun (veya km gelmedi), iş emri güncel (≤5 gün)	Mevcut iş emrine işlenir; yeni iş emri açılmaz
Aynı `kayit_no` tekrar	Yinelenmez, `tekrar:true` döner
Plaka kayıtsız / açık iş emri yok (örn. iş emri zaten Tamamlandı) / birden fazla açık iş emri / km uyuşmuyor / iş emri çok eski	202 — inceleme listesi, manuel eşleştirme
`yapilan_islem` mevcut metinden farklı	Yeni metin yazılır; eski metin nota "Önceki işlem" olarak saklanır

Tetikleme önerisi: Bildirim her muhasebe hareketinde (ödeme, fatura satırı) değil, servis/iş emri kaydı oluştuğunda tek olay olarak gönderilmelidir. Aynı servis için birden fazla belge oluşuyorsa her biri farklı `kayit_no` ile gönderilir. `durum: Tamamlandı` gönderilen iş emri kapanır; sonraki kayıtlar incelemeye alınır.

8. Güvenilirlik ve güvenlik

- **Idempotency:** `kayit_no` ile tekrar gönderim güvenlidir.
- **Kalıcılık:** Yanıt (200/202) dönmeden önce kayıt veritabanına yazılır.
- **Veri bütünlüğü:** Karşı sistemin veritabanına erişilmez; yalnızca gönderilen bilgi işlenir. Müşteri ve araç bilgileri yalnızca boşsa tamamlanır, ezilmez.
- **Erişim:** HTTPS, API anahtarı, hatalı deneme sınırlaması; anahtar sızarsa yenisi üretilir.

9. Entegrasyon test planı

Canlı verilerle denemeden önce sırayla şu adımları uygulayın. Her adımın beklenen sonucu yanındadır.

#	Adım	Beklenen
1	`GET /saglik`	200, `{"ok":true,…}` — bağlantı ve TLS çalışıyor
2	Anahtarsız POST	401 `INVALID_API_KEY` — güvenlik açık
3	Doğru anahtar, plakasız gövde	422 `VALIDATION_ERROR` — anahtar kabul edildi, doğrulama çalışıyor
4	Kayıtlı olmayan bir test plakası ile POST	202 `beklemede` — kayıt güvenle saklandı (Servis Defteri'nde "Eşleşmeyen" listesinde görünür)
5	Atölye, test plakası ile bir iş emri açsın; ardından plaka + km (+ `yapilan_islem`, `durum`) gönderin	200 `eslesti:true`; atölyede aynı iş emrinde "Yapılan işlem" ve durum güncellenmiş görünür
6	Aynı isteği aynı `kayit_no` ile tekrar gönderin	200 `tekrar:true` — çift kayıt oluşmaz

3. adımda 403 alırsanız bkz. Bölüm 3 (User-Agent). Sorunda isteğin zamanını, `kayit_no` değerini ve aldığınız yanıtı iletirseniz nedeni hızlıca bulunabilir.

10. Sürümleme ve destek

Bu belge v1.1'i tanımlar. Geriye uyumlu eklemeler (yeni isteğe bağlı alanlar) sürüm değiştirmez. Makine-okunur tanım: `openapi.yaml` (OpenAPI 3.0). Destek için Servis Defteri'ni işleten işletmeyle iletişime geçin.

11. Örnek kodlar

Aşağıdaki örneklerde `API_ANAHTARI` yerine size iletilen anahtar yazılır. Örnekler aynı isteği gönderir; 200 / 202 başarılı sayılır. Hepsinde User-Agent başlığı bulunur. Aynı dosyalar `docs/ornekler/` klasöründedir.

curl

```
#!/bin/sh
# curl ornegi
BASE="https://alanadiniz.com/api/v1"
KEY="API_ANAHTARI"
curl -sS -X POST "$BASE/is-emirleri" \
  -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -H "User-Agent: MuhasebeYazilimi/1.0" \
  -d '{"kayit_no":"K-10425","plaka":"35 ABC 123","km":120010,"yapilan_islem":["Yağ
değişimi","Yağ filtresi"],"durum":"Tamamlandı"}'
```

Python 3

```
# Python 3 ornegi (ek paket gerekmez)
import json, time, urllib.request, urllib.error

BASE = "https://alanadiniz.com/api/v1"
KEY = "API_ANAHTARI"

def bildir(kayit, deneme=3):
    istek = urllib.request.Request(BASE + "/is-emirleri", data=json.dumps(kayit).encode("utf-
8"), method="POST",
                                   headers={"Authorization": "Bearer " + KEY, "Content-Type":
"application/json", "User-Agent": "MuhasebeYazilimi/1.0"})
    for n in range(deneme):
        try:
            with urllib.request.urlopen(istek, timeout=20) as r:
                return r.status, json.loads(r.read().decode())          # 200 eslesti/tekrar,
202 incelemeye alindi
        except urllib.error.HTTPError as e:
            govde = json.loads(e.read().decode() or "{}")
            if e.code == 429: time.sleep(int(e.headers.get("Retry-After", "300"))); continue
            if e.code >= 500 or (e.code == 409 and govde.get("kod") == "CONFLICT"): time.sleep(2
** n); continue   # gecici: ayni kayit_no ile tekrar
            return e.code, govde                                        #
401/422/409(IDEMPOTENCY_CONFLICT): duzeltmeden tekrar denemeyin
        except OSError: time.sleep(2 ** n)
    return 0, {"hata": "ulasilamadi"}
```

```
if __name__ == "__main__":
    print(bildir({"kayit_no": "K-10425", "plaka": "35 ABC 123", "km": 120010, "yapilan_islem":
["Yağ değişimi"], "durum": "Tamamlandı"}))
```

Node.js (18+)

```
// Node.js 18+ ornegi (fetch yerlesik)
const BASE = "https://alanadiniz.com/api/v1";
const KEY = "API_ANAHTARI";

async function bildir(kayit) {
  const r = await fetch(BASE + "/is-emirleri", {
    method: "POST",
    headers: { "Authorization": "Bearer " + KEY, "Content-Type": "application/json", "User-
Agent": "MuhasebeYazilimi/1.0" },
    body: JSON.stringify(kayit),
  });
  return { durum: r.status, govde: await r.json() };   // 200 eslesti/tekrar, 202 incelemeye
alindi
}

bildir({ kayit_no: "K-10425", plaka: "35 ABC 123", km: 120010, yapilan_islem: ["Yağ değişimi"],
durum: "Tamamlandı" })
  .then(console.log).catch(console.error);
```

PowerShell

```
# PowerShell 5.1+ ornegi
$Base = "https://alanadiniz.com/api/v1"
$Key  = "API_ANAHTARI"
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12   # eski
sistemlerde TLS 1.2 gerekir

$kayit = @{ kayit_no = "K-10425"; plaka = "35 ABC 123"; km = 120010; yapilan_islem = @("Yağ
değişimi", "Yağ filtresi"); durum = "Tamamlandı" }
$govde = [Text.Encoding]::UTF8.GetBytes(($kayit | ConvertTo-Json -Compress))
try {
    $yanit = Invoke-RestMethod -Method Post -Uri "$Base/is-emirleri" -ContentType
"application/json; charset=utf-8" `
             -Headers @{ Authorization = "Bearer $Key"; "User-Agent" = "MuhasebeYazilimi/1.0" }
-Body $govde
    $yanit | ConvertTo-Json            # eslesti=true -> 200, durum=beklemede -> 202
} catch {
    $kod = [int]$_.Exception.Response.StatusCode
    "HTTP $kod : " + $_.ErrorDetails.Message   # 401/422/409 duzeltmeden tekrar denenmez; 5xx
tekrar denenir
}
```

C# (.NET)

```
// C# (.NET 6+ / .NET Framework 4.7.2+) ornegi
using System;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Text;
using System.Threading.Tasks;
```

```
class Program
{
    const string Base = "https://alanadiniz.com/api/v1";
    const string Key = "API_ANAHTARI";

    static async Task Main()
    {
        using var http = new HttpClient();
        http.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", Key);
        http.DefaultRequestHeaders.UserAgent.ParseAdd("MuhasebeYazilimi/1.0");   // User-Agent
gonderin; bos/varsayilan kimlikler guvenlik duvarinca engellenebilir
        var json = "{\"kayit_no\":\"K-10425\",\"plaka\":\"35 ABC
123\",\"km\":120010,\"yapilan_islem\":[\"Yağ değişimi\"],\"durum\":\"Tamamlandı\"}";
        var yanit = await http.PostAsync(Base + "/is-emirleri", new StringContent(json,
Encoding.UTF8, "application/json"));
        Console.WriteLine((int)yanit.StatusCode + " " + await
yanit.Content.ReadAsStringAsync());
        // 200 eslesti/tekrar, 202 incelemeye alindi -> basarili sayin. 5xx ve 429 (Retry-After)
tekrar denenir.
    }
}
```